

Browser Security Model

Mainack Mondal

CS60112

Spring 2025



Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

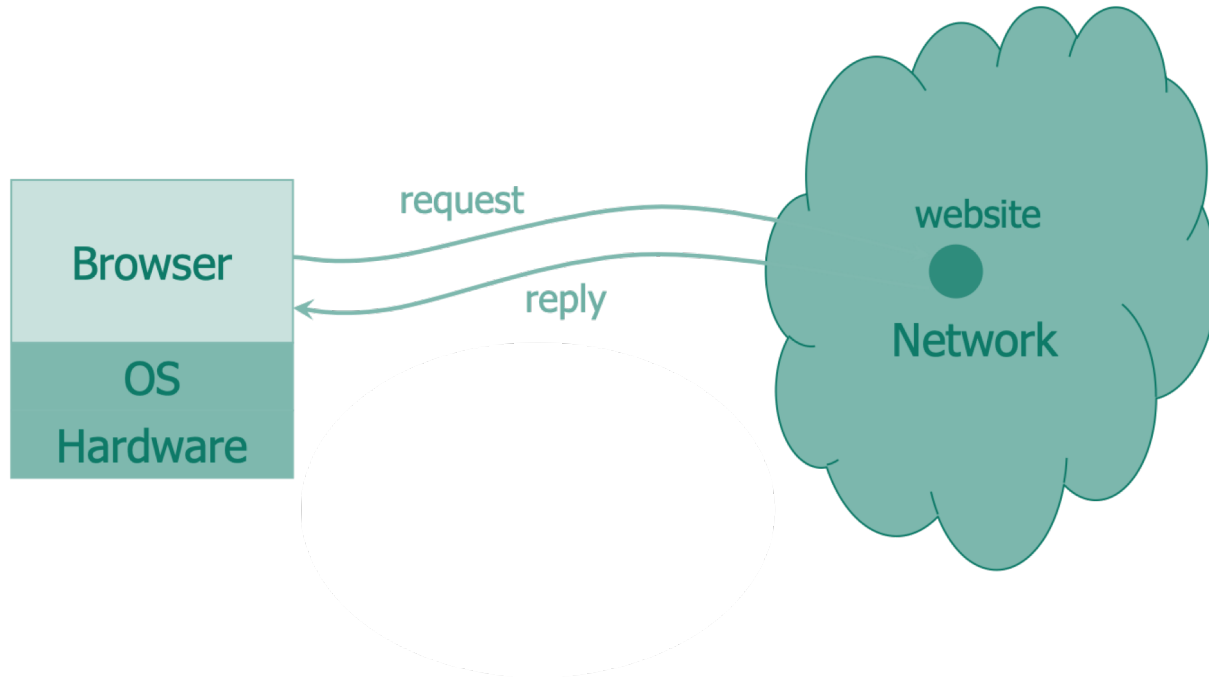
Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

Slides partially taken from John Mitchell, CS155, Stanford

WEB SECURITY MODEL

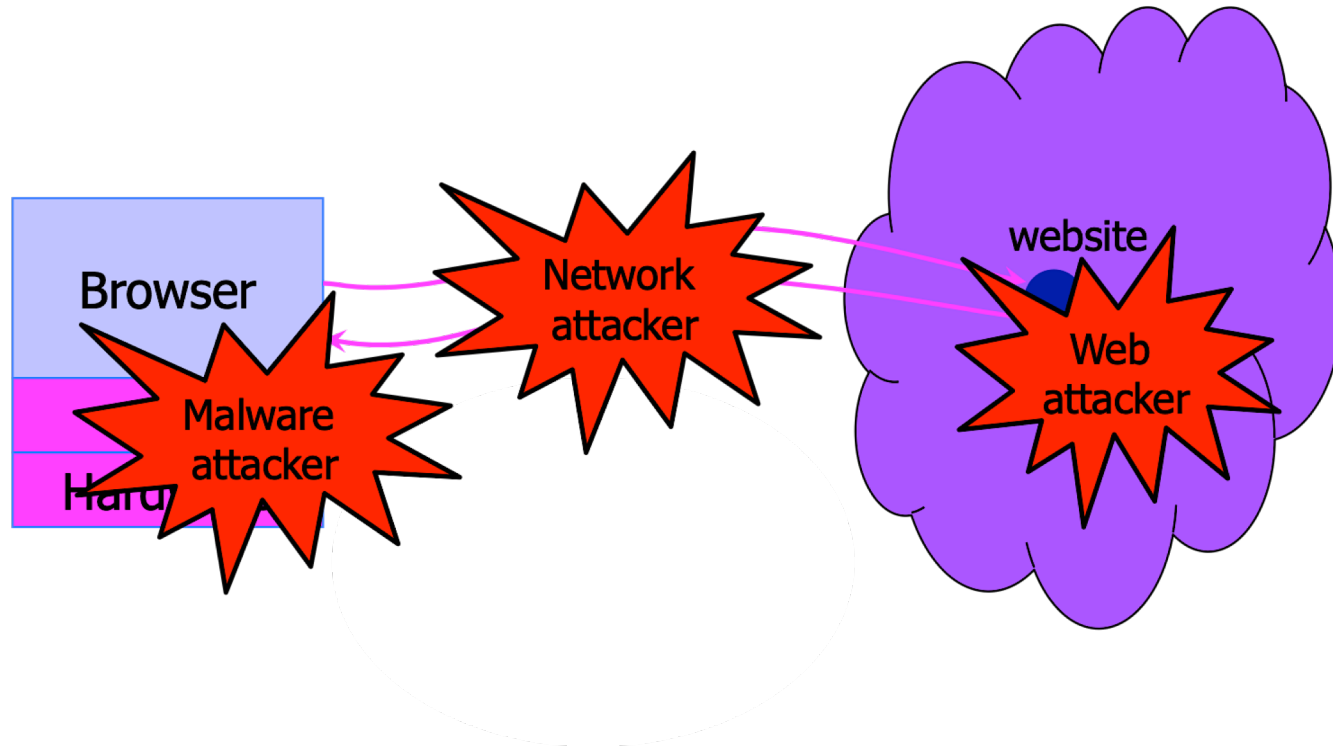
Browser and network



Web Security

- Web browser
 - Responsible for securely confining Web content presented by visited websites
- Web applications
 - Online merchants, banks, blogs, Google Apps ...
 - Mix of server-side and client-side code
 - Server-side code written in PHP, Ruby, ASP, JSP... runs on the Web server
 - Client-side code written in JavaScript... runs in the Web browser
- Many potential bugs: XSS, XSRF, SQL injection,...

Where is the attacker?



What would the attacker do?

- Controls a malicious website (attacker.com)
 - Can even obtain an SSL/TLS certificate for his site (\$0)
- User visits attacker.com
 - Phishing email, enticing content, search results, placed by an ad network, blind luck ...
 - Attacker's Facebook app
- Attacker has no other access to user machine!
- Variation: "iframe attacker"
 - An iframe with malicious content included in an otherwise honest webpage
 - Syndicated advertising, mashups, etc.

Network and malware attacker

- Network attacker

- Passive: wireless eavesdropper
- Active: evil Wi-Fi router, DNS poisoning

- Malware attacker

- Malicious code executes directly on victim's computer
- To infect victim's computer, can exploit software bugs (e.g., buffer overflow) or convince user to install malicious content
 - Masquerade as an antivirus program, video codec, etc.

Goals of web security

- Safely browse the Web
 - A malicious website cannot steal information from or modify legitimate sites or otherwise harm the user...
 - ... even if visited concurrently with a legitimate site - in a separate browser window, tab, or even iframe on the same webpage
- Support secure Web applications
 - Applications delivered over the Web should have the same security properties as required for standalone applications
 - which ones? Anonymity, authenticity, secrecy...very application dependent!

Policy Goals

- Safe to visit an evil web site
- Safe to visit two pages at the same time
 - Address bar distinguishes them
- Safe loading of content



OS vs. browser analogies

Operating System

- Primitives
 - System calls
 - Processes
 - Disk
- Principals: Users
 - Discretionary access control
- Vulnerabilities
 - Buffer overflow
 - Root exploit

Web Browser

- Primitives
 - Document object model
 - Frames
 - Cookies and local storage
- Principals: Origins
 - Mandatory access control
- Vulnerabilities
 - Cross-site scripting (XSS)
 - Universal XSS
 - exploits vulnerabilities in the browser, rather than in a website

Browser: basic execution model

- Each browser window or frame:
 - Loads content
 - Renders
 - Processes HTML and scripts to display the page
 - May involve images, subframes, etc.
 - Responds to events
- Events
 - User actions: OnClick, OnMouseover
 - Rendering: OnLoad, OnUnload
 - Timing: setTimeout(), clearTimeout()

Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

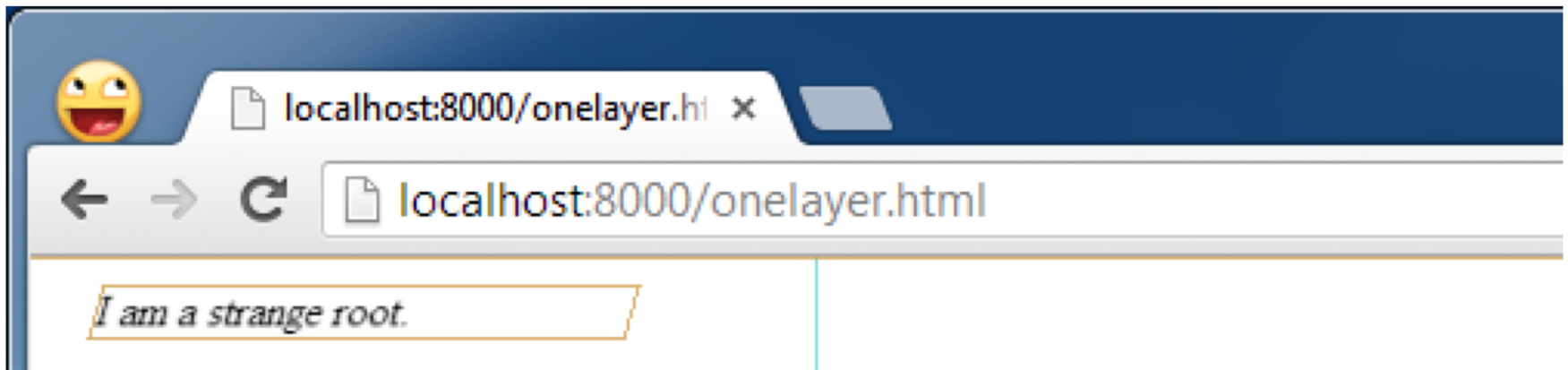
RENDERING CONTENT

Rendering and events

- Basic browser execution model
 - Each browser window or frame
 - Loads content
 - Renders it
 - Processes HTML and scripts to display page
 - May involve images, subframes, etc.
 - Responds to events
- Events can be
 - User actions: OnClick, OnMouseover
 - Rendering: OnLoad, OnBeforeUnload
 - Timing: setTimeout(), clearTimeout()

Example

```
<html>
  <body>
    <div style="-webkit-transform: rotateY(30deg)
      rotateX(-30deg); width: 200px;">
      I am a strange root.
    </div>
  </body>
</html>
```



Document Object Model (DOM)

- Object-oriented interface used to read and write docs
 - web page in HTML is structured data
 - DOM provides representation of this hierarchy
- Examples
 - **Properties:** document.alinkColor, document.URL, document.forms[], document.links[], document.anchors[]
 - **Methods:** document.write(document.referrer)
- Includes Browser Object Model (BOM)
 - window, document, frames[], history, location, navigator (type and version of browser)

Changing HTML using Script, DOM

HTML

- Some possibilities
 - createElement(elementName)
 - createTextNode(text)
 - appendChild(newChild)
 - removeChild(node)
- Example: Add a new list item:

```
<ul id="t1">  
<li> Item 1 </li>  
</ul>
```

```
var list = document.getElementById('t1')  
var newitem = document.createElement('li')  
var newtext = document.createTextNode(text)  
list.appendChild(newitem)  
newitem.appendChild(newtext)
```

HTML Image Tags

```
<html>  
...  
<p> ... </p>  
...  
  
...  
</html>
```

Displays this nice picture →
Security issues?



Image tag security issues

- Communicate with other sites
 - ``
- Hide resulting image
 - ``
- Spoof other sites
 - Add logos that fool a user

Important Point: A web page can send information to any site

JavaScript onError

- Basic function
 - Triggered when error occurs loading a document or an image
- Example

```

```

- Runs onError handler if image does not exist and cannot load

http://www.w3schools.com/jsref/jsref_onError.asp

An application

- What will happen?

```
<html><body><img id="test" style="display: none">
<script>
  var test = document.getElementById('test');
  var start = new Date();
  test.onerror = function() {
    var end = new Date();
    alert("Total time: " + (end - start));
  }
  test.src = "http://www.example.com/page.html";
</script>
</body></html>
```

Measuring JavaScript timing with onError

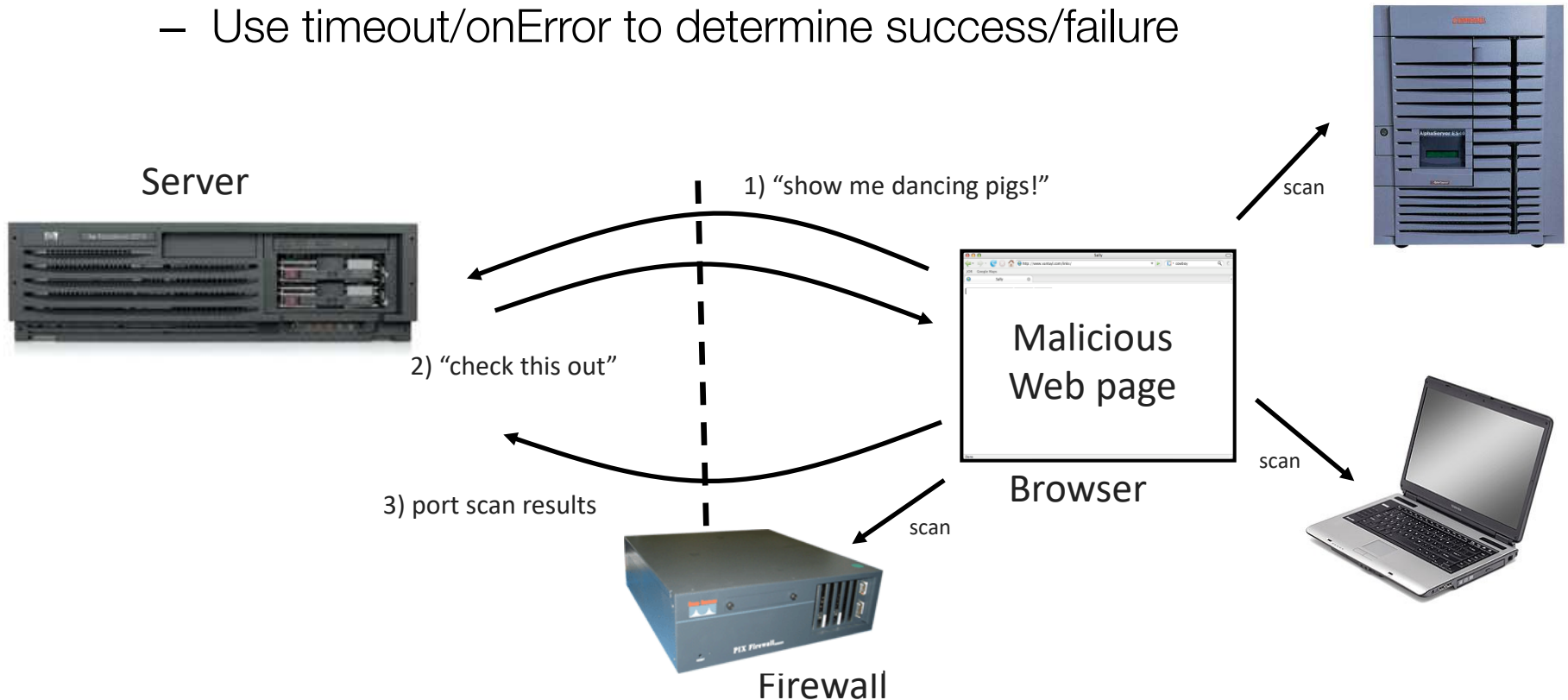
- What will happen?

```
<html><body><img id="test" style="display: none">
<script>
  var test = document.getElementById('test');
  var start = new Date();
  test.onerror = function() {
    var end = new Date();
    alert("Total time: " + (end - start));
  }
  test.src = "http://www.example.com/page.html";
</script>
</body></html>
```

- When response header indicates that page is not an image, the browser stops and notifies JavaScript via the onerror handler.

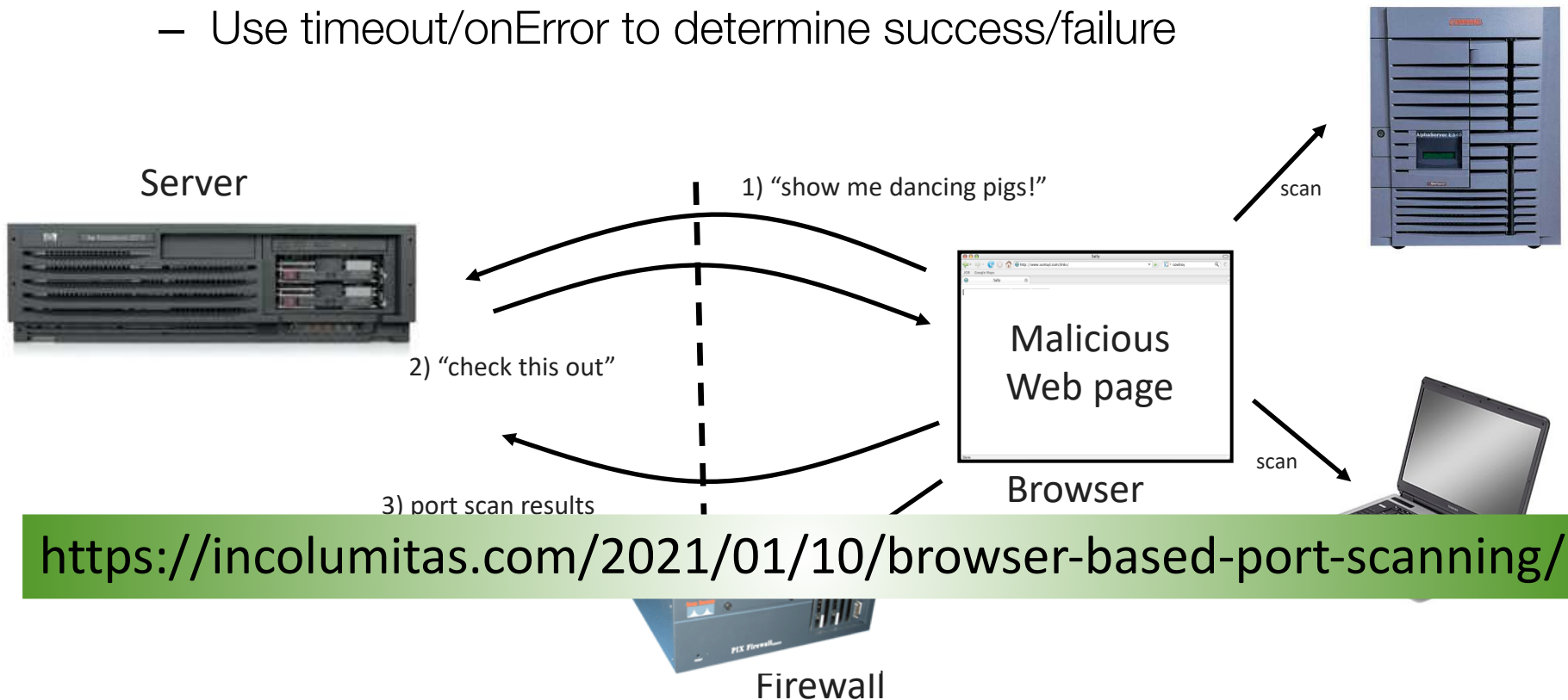
Port scanning behind firewall

- JavaScript can:
 - Request images from internal IP addresses
 - Example: ``
 - Use timeout/onError to determine success/failure



Port scanning behind firewall

- JavaScript can:
 - Request images from internal IP addresses
 - Example: ``
 - Use timeout/onError to determine success/failure



<https://incolumitas.com/2021/01/10/browser-based-port-scanning/>

Remote scripting

- Goal
 - Exchange data between a client-side app running in a browser and server-side app, without reloading page
- Methods
 - Java Applet/ActiveX control/Flash
 - Can make HTTP requests and interact with client-side JavaScript code, but requires LiveConnect (not available on all browsers)
 - XML-RPC
 - open, standards-based technology that requires XML-RPC libraries on server and in your client-side code.
 - Simple HTTP via a hidden IFRAME
 - IFRAME with a script on your web server (or database of static HTML files) is by far the easiest of the three remote scripting options

Important Point: A page can maintain bi-directional communication with browser (until user closes/quits)

See: <http://developer.apple.com/internet/webcontent/iframe.html>

More remote scripting

- Goal
 - Exchange data between a client-side app running in a browser and server-side app, without reloading page
- Methods
 - XMLHttpRequest
 - Ajax
 - Asynchronous JavaScript And XML.
 - practical use of the XMLHttpRequest to communicate with servers.

Important Point: A page can maintain bi-directional communication with browser (until user closes/quits)

See: <http://developer.apple.com/internet/webcontent/iframe.html>

Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

ISOLATION

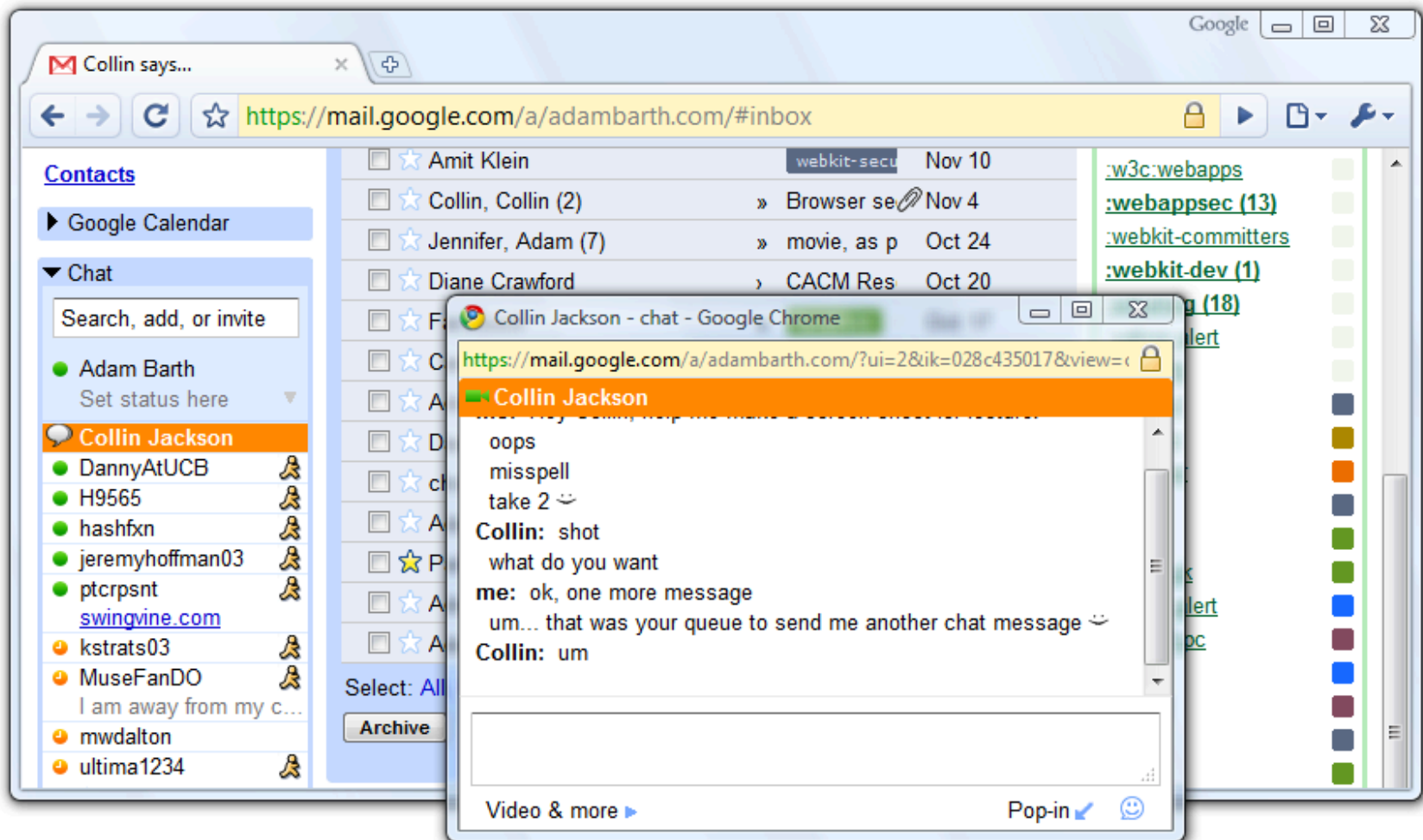
Frame and iFrame

- Window may contain frames from different sources
 - Frame: rigid division as part of frameset
 - iFrame: floating inline frame
- iFrame example

```
<iframe src="hello.html" width=450 height=100>  
If you can see this, your browser doesn't understand IFRAME.  
</iframe>
```

- Why use frames?
 - Delegate screen area to content from another source
 - Browser provides isolation based on frames
 - Parent may work even if frame is broken

Windows Interact



Policy Goals

- Safe to visit an evil web site



- Safe to visit two pages at the same time

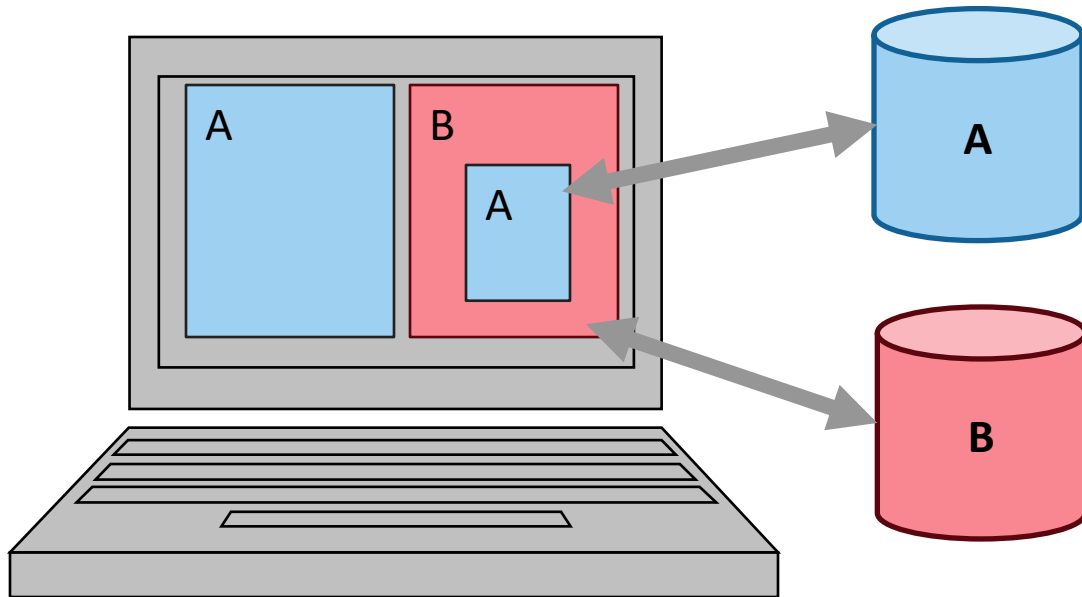
- Address bar distinguishes them



- Allow safe delegation



Browser security mechanism



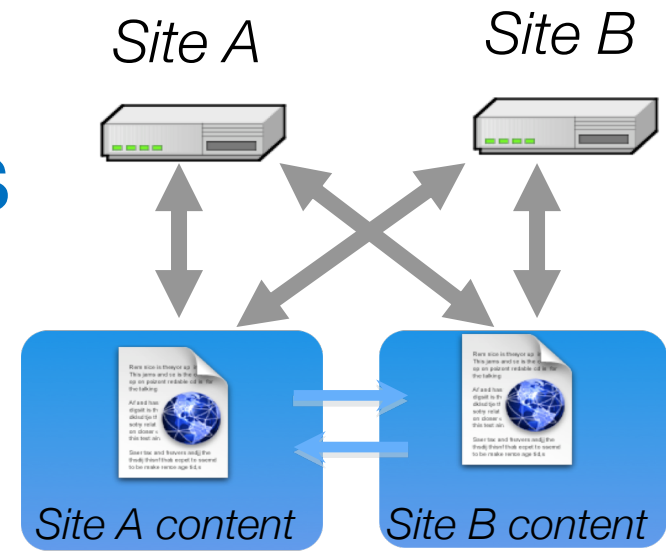
- Each frame of a page has an origin
 - Origin = protocol://host:port
- Frame can access its own origin
 - Network access, Read/write DOM, Storage (cookies)
- Frame cannot access data associated with a different origin

Components of browser security policy

- Frame-Frame relationships
 - canScript(A,B)
 - Can Frame A execute a script that manipulates arbitrary/nontrivial DOM elements of Frame B?
 - canNavigate(A,B)
 - Can Frame A change the origin of content for Frame B?
- Frame-principal relationships
 - readCookie(A,S), writeCookie(A,S)
 - Can Frame A read/write cookies from site S?

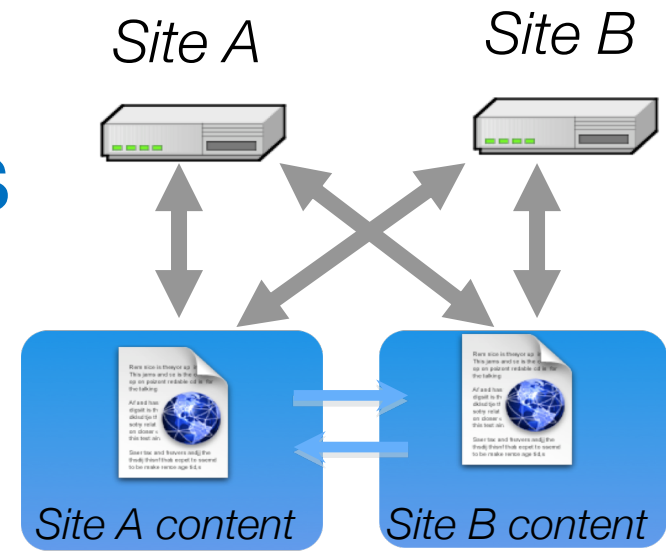
See <https://code.google.com/p/browsersec/wiki/Part1>
<https://code.google.com/p/browsersec/wiki/Part2>
https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

Additional mechanisms



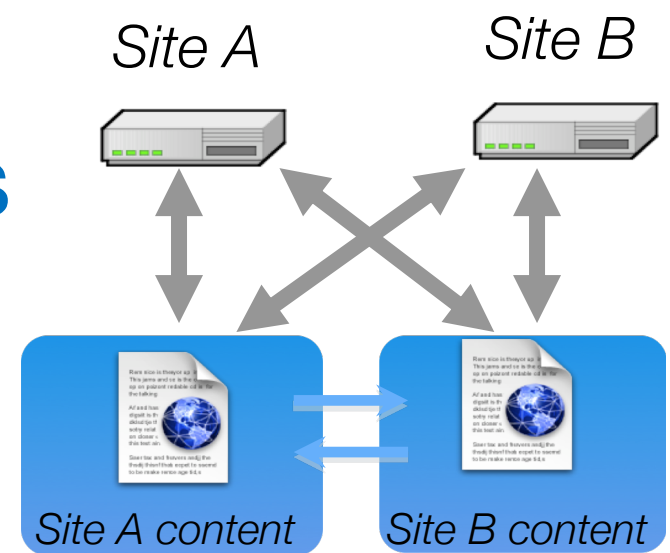
- ❑ Cross-origin resource sharing
 - ❑ Response header, allow a file to be requested
 - ❑ Access-Control-Allow-Origin: <list of domains>
 - ❑ Access-Control-Allow-Origin: *
- ❑ Cross-origin client side communication
 - ❑ Using same document.domain property
 - ❑ postMessage
 - ❑ Websockets

Additional mechanisms



- ❑ Cross-origin resource sharing
 - ❑ Response header, allow a file to be requested
 - ❑ Access-Control-Allow-Origin: <list of domains>
 - ❑ Access-Control-Allow-Origin: *
- ❑ Cross-origin client side communication
 - ❑ Using same document.domain property
 - ❑ postMessage ?
 - ❑ Websockets ?

Additional mechanisms



- ❑ Cross-origin resource sharing
 - ❑ Response header, allow a file to be requested
 - ❑ Access-Control-Allow-Origin: <list of domains>
 - ❑ Access-Control-Allow-Origin: *
- ❑ Cross-origin client side communication
 - ❑ Using same document.domain property
 - ❑ postMessage ?
 - ❑ Websockets ?

Read

1. <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
2. <https://appcheck-ng.com/html5-cross-document-messaging-vulnerabilities/>
3. <https://medium.com/@sergey.dudik/mastering-real-time-communication-a-comprehensive-websocket-tutorial-0f6cf384d1e8>

Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

COMMUNICATION

window.postMessage

- API for inter-frame communication
 - Supported in many browsers

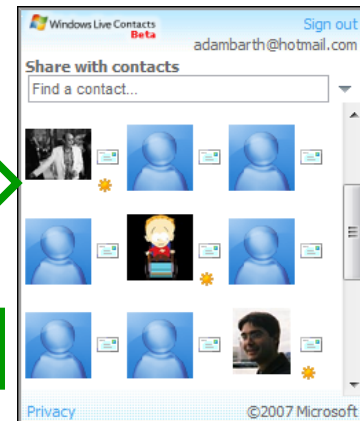


- A network-like channel between frames



Add a contact

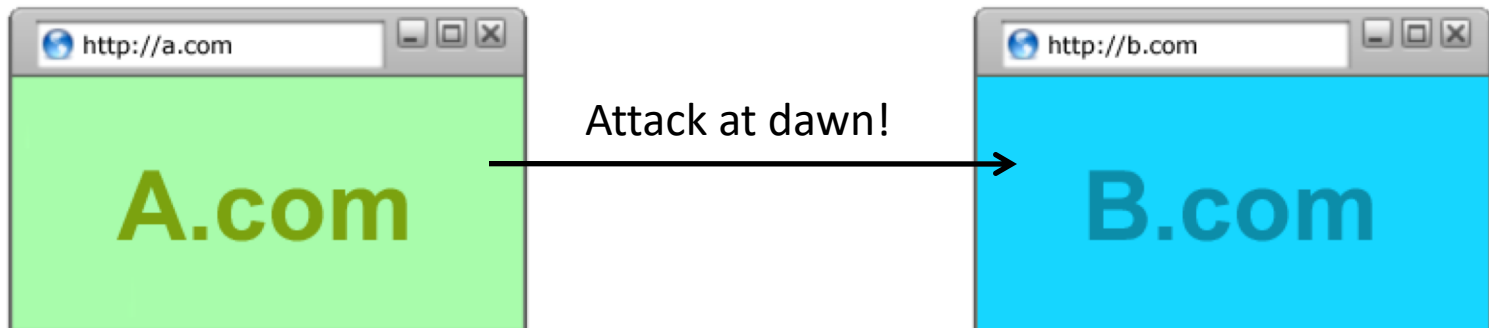
Share contacts



postMessage syntax

```
frames[0].postMessage("Attack at dawn!",  
                      "http://b.com/");
```

```
window.addEventListener("message", function (e) {  
  if (e.origin == "http://a.com") { ← Verify origin  
    ... e.data ... }  
}, false);
```



Why include “targetOrigin”?

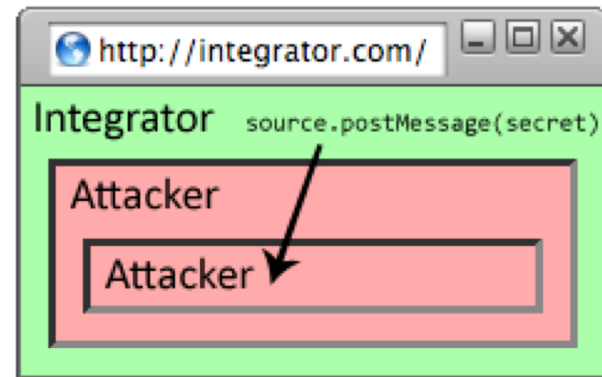
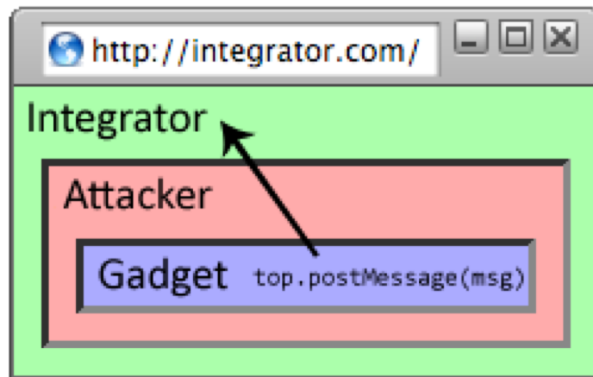
- What goes wrong?

```
frames[0].postMessage("Attack at dawn!");
```


Why include “targetOrigin”?

- What goes wrong?

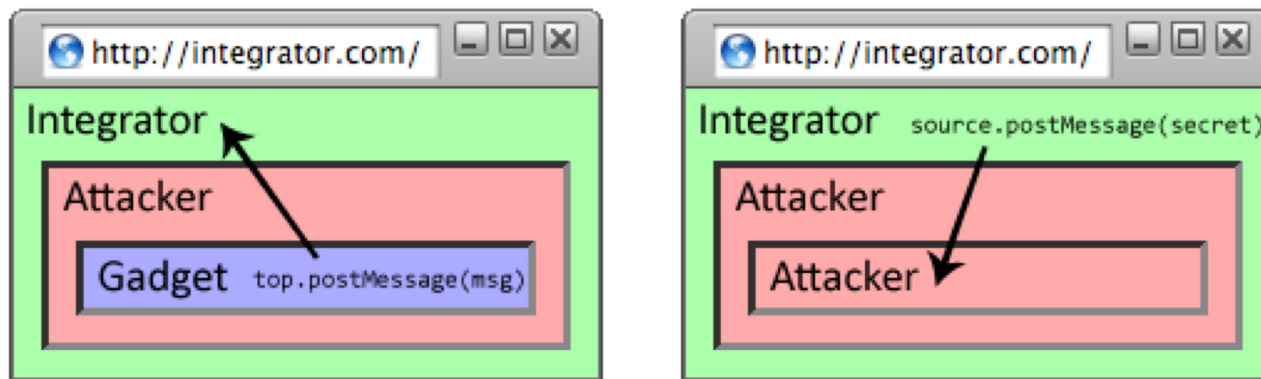
```
frames[0].postMessage("Attack at dawn!");
```



Why include “targetOrigin”?

- What goes wrong?

```
frames[0].postMessage("Attack at dawn!");
```



When you are done with attacks (next week), go over:

1. <https://gist.github.com/jedp/3005816>
2. <https://seclab.stanford.edu/websec/frames/post-message.pdf>
3. <https://www.yeswehack.com/learn-bug-bounty/introduction-postmessage-vulnerabilities>
4. https://docs.ioin.in/writeup/www.exploit-db.com/_docs_40287_pdf/index.p

Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

NAVIGATION

Navigation: Idea

- Who is allowed to decide what content appears in a frame?

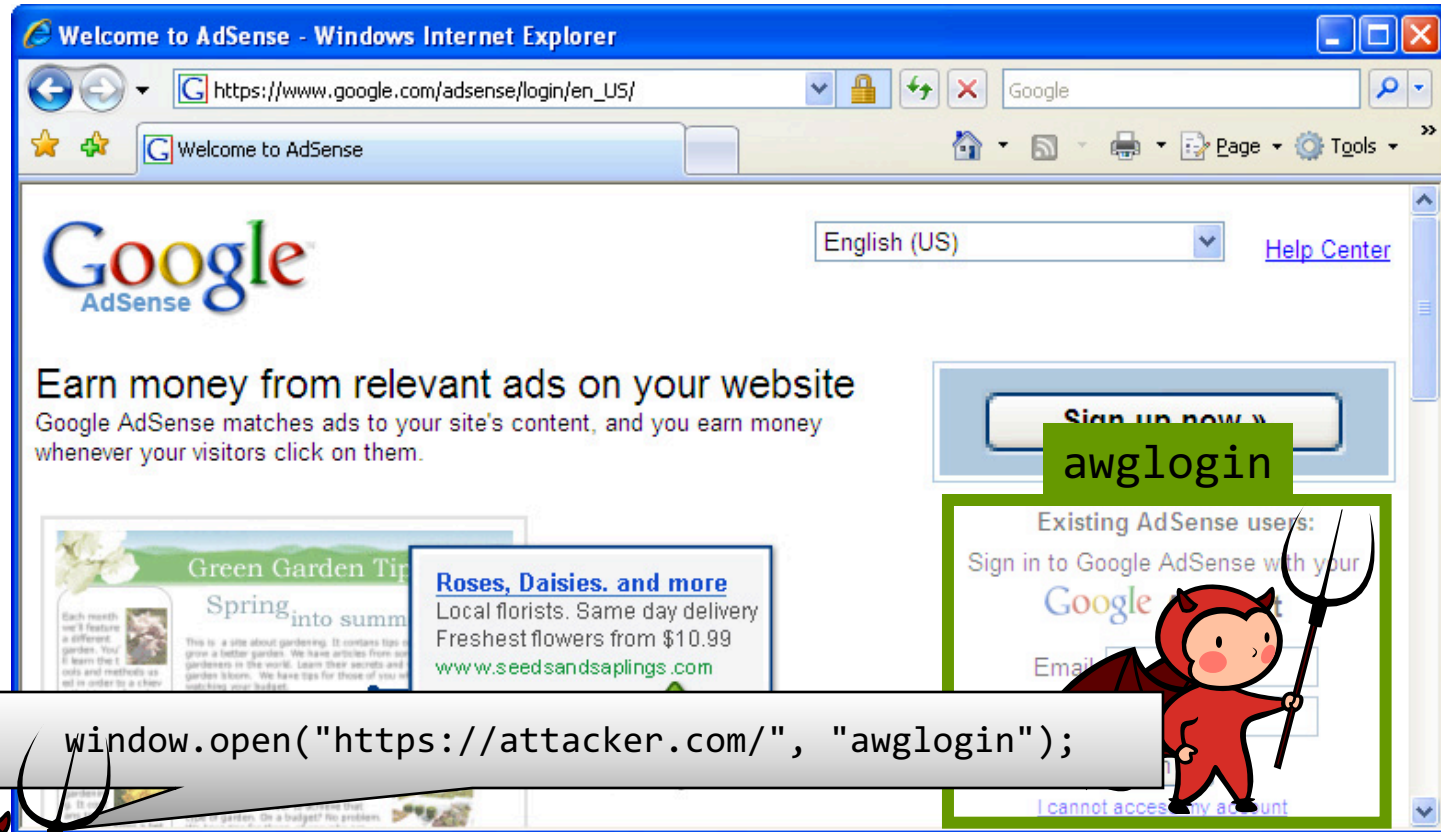
```
frame.src = "http://www.attacker.com"
```

- Original policy was permissive: any frame could navigate any other frame
- Guninski attack
 - Password field on Citibank website contained within a frame
 - Attacker could navigate this frame to an identical looking one owned by the attacker; steals password

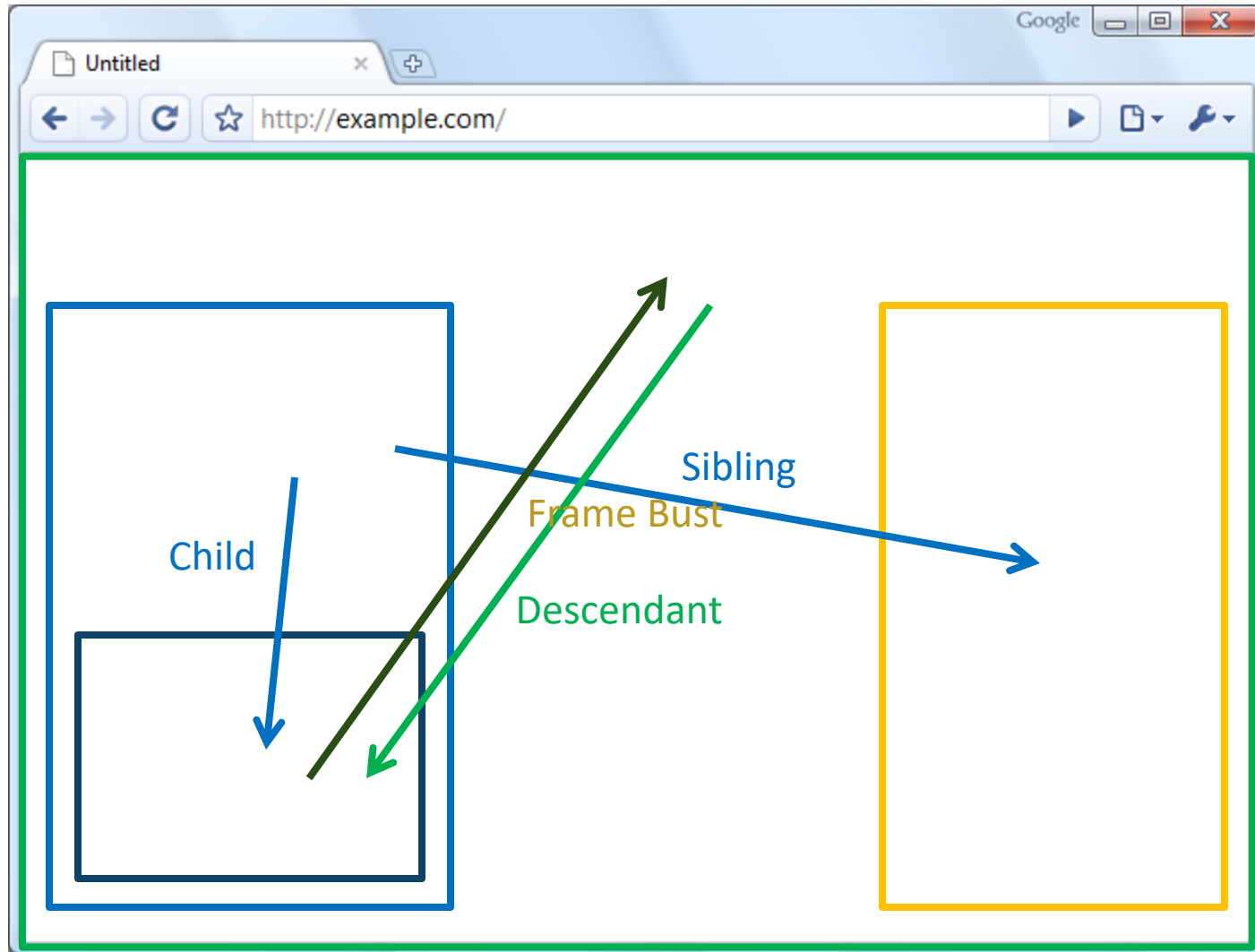
```
open("http://www.attacker.com/citibank", "password")
```

- Attack can come from any open window or tab

Guninski Attack:Example



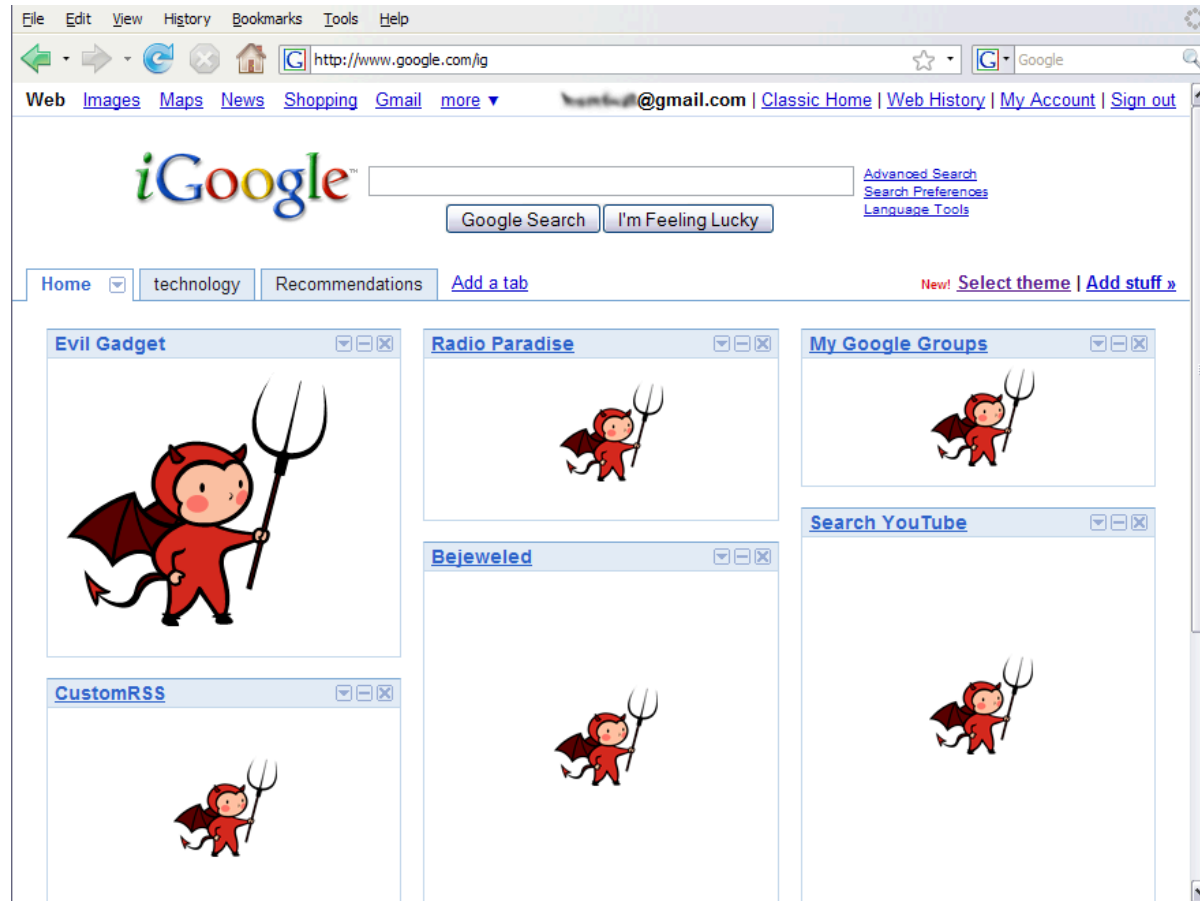
What should the policy be?



Legacy Browser Behavior

Browser	Policy
 IE 6 (default)	Permissive
 IE 6 (option)	Child
 IE7 (no Flash)	Descendant
 IE7 (with Flash)	Permissive
 Firefox 2	Window
 Safari 3	Permissive
 Opera 9	Window
 HTML 5	Child

Window Policy Anomaly



.com/...";
.com/...";

Solution

- Descendent policy: A frame can only navigate its children , and their children, etc
 - Most current browsers implement this policy

Legacy Browser Behavior

Browser	Policy
 IE 6 (default)	Permissive
 IE 6 (option)	Child
 IE7 (no Flash)	Descendant
 IE7 (with Flash)	Permissive
 Firefox 2	Window
 Safari 3	Permissive
 Opera 9	Window
 HTML 5	Child

Adoption of Descendant Policy

Browser	Policy
 IE7 (no Flash)	Descendant
 IE7 (with Flash)	Descendant
 Firefox 3	Descendant
 Safari 3	Descendant
 Opera 9	(many policies)
 HTML 5	Descendant

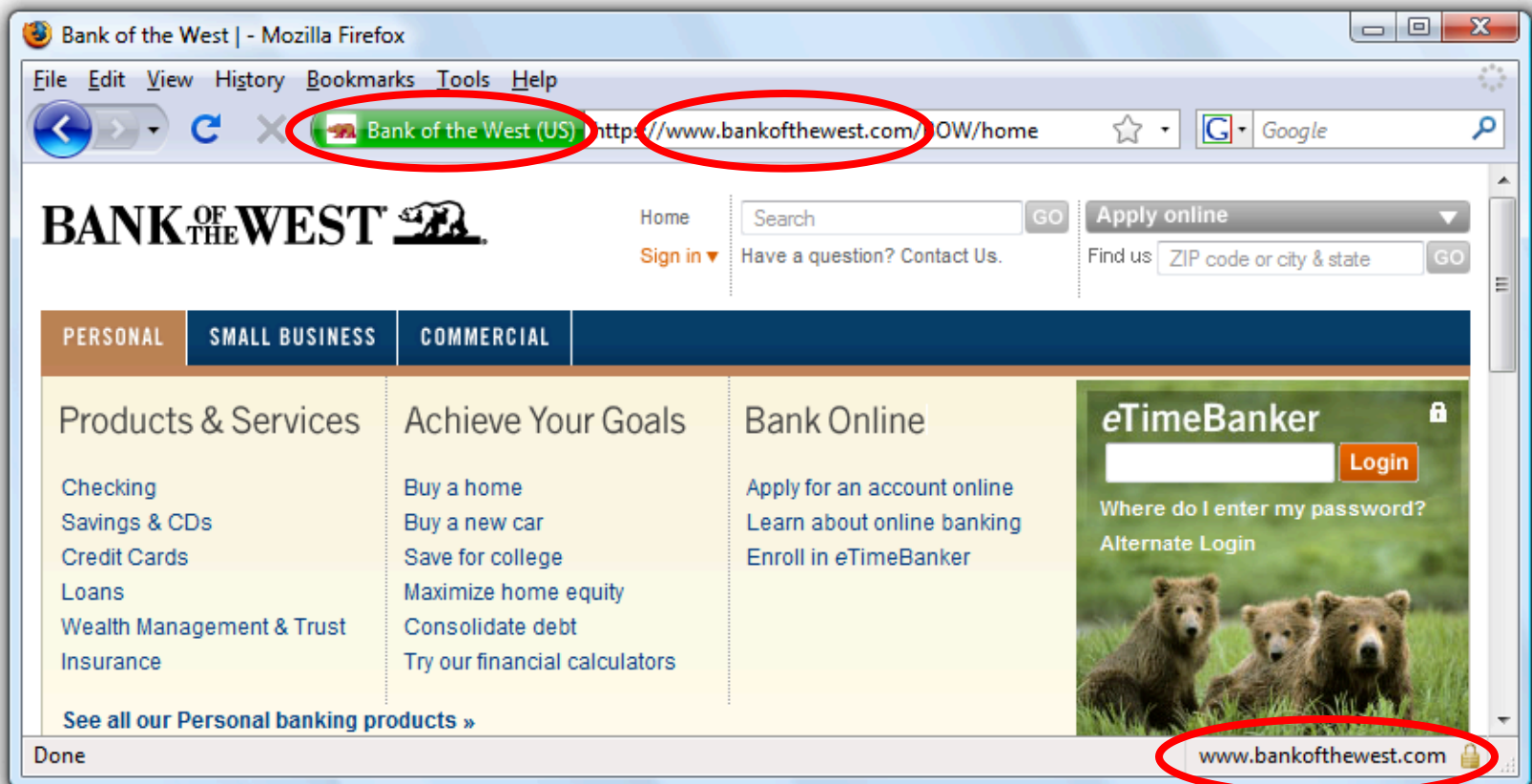
Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

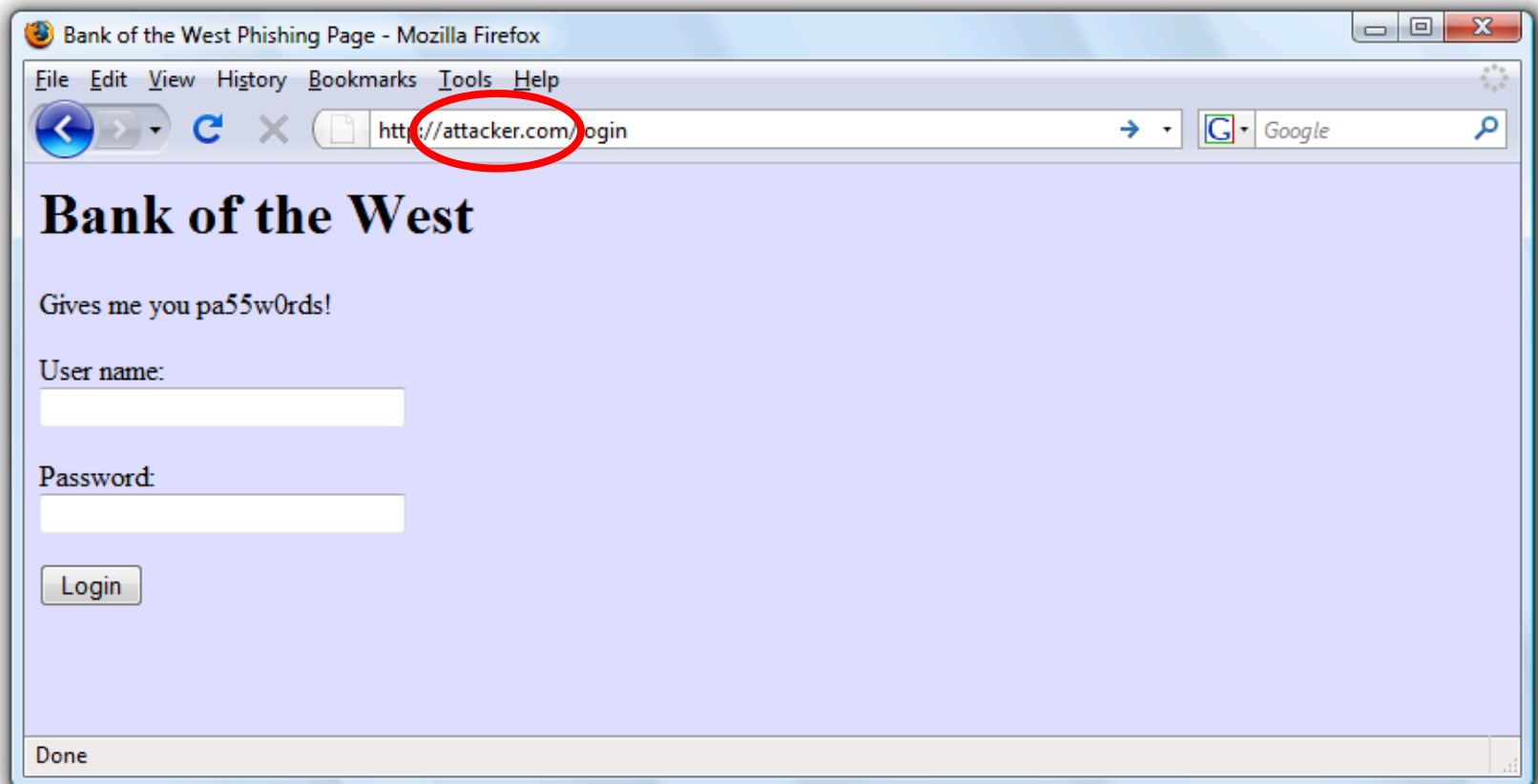
When is it safe to type my password?

SECURITY USER INTERFACE

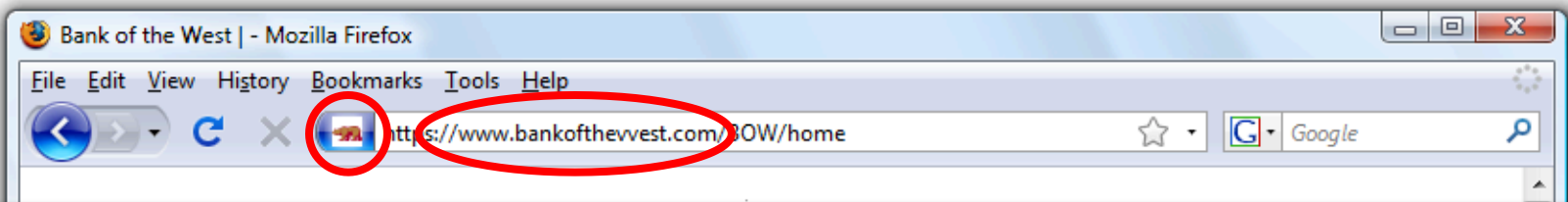
Safe to type your password?



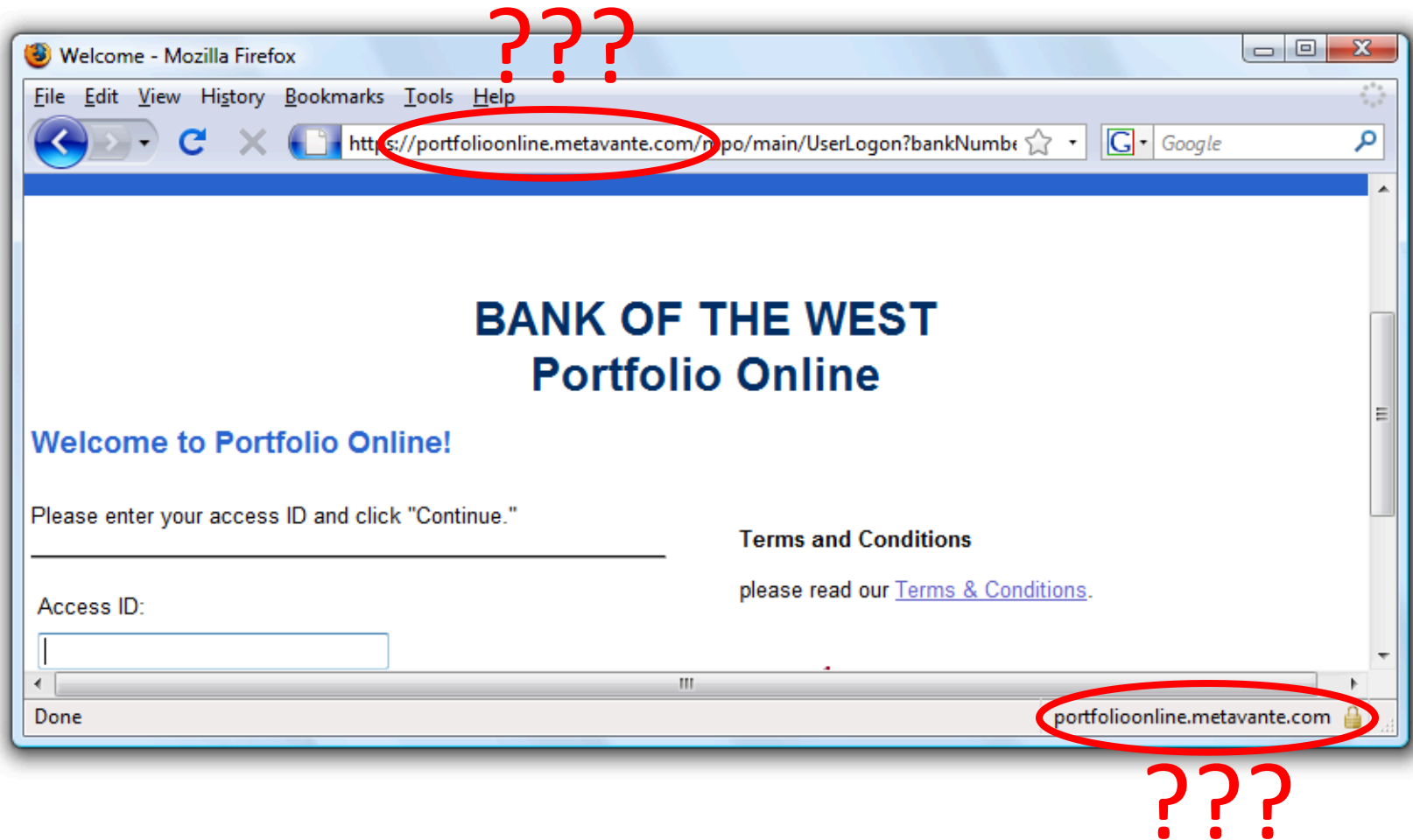
Safe to type your password?



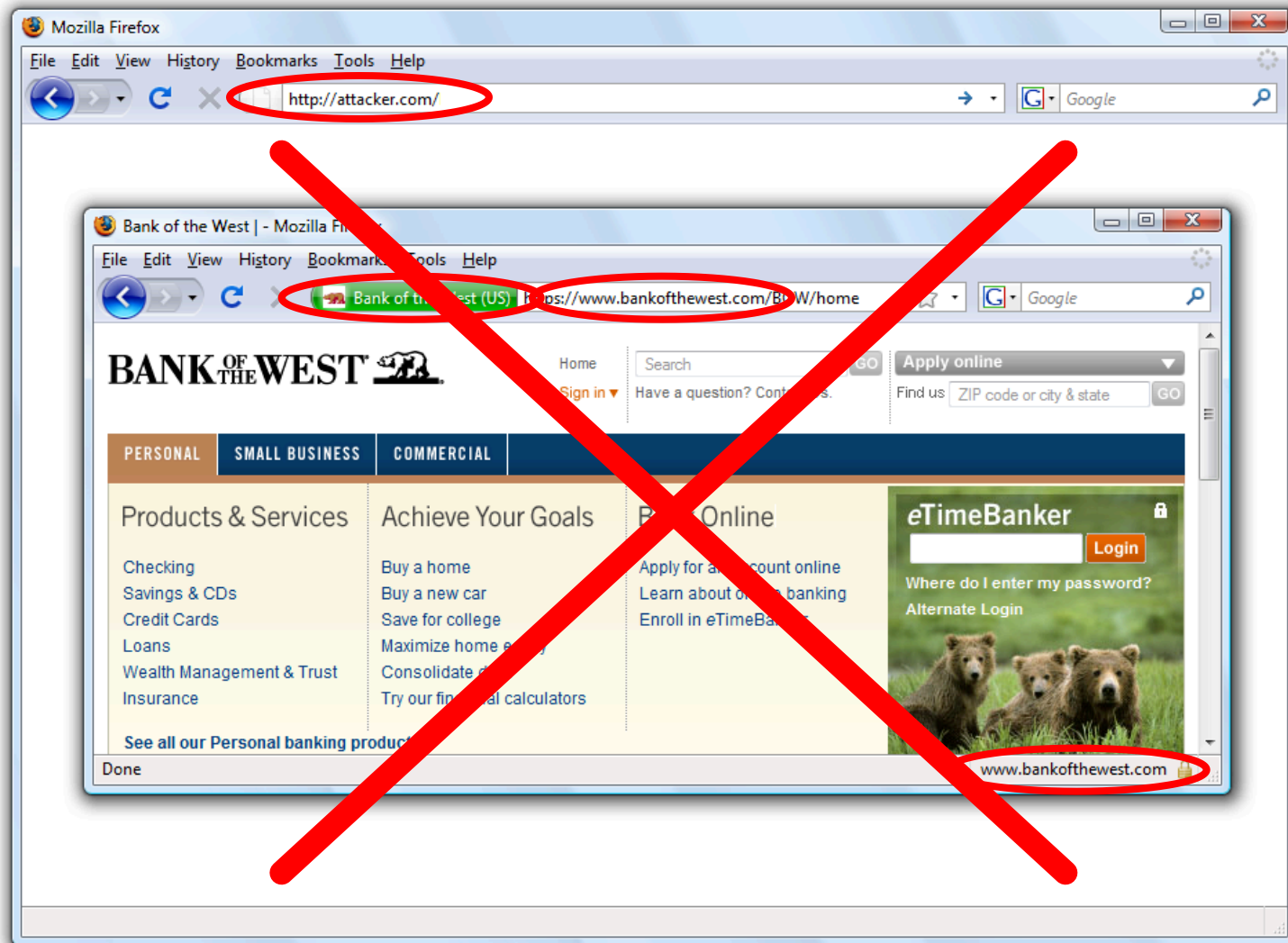
Safe to type your password?



Safe to type your password?



Safe to type your password?



Mixed Content: HTTP and HTTPS

- Problem
 - Page loads over HTTPS, but has HTTP content
 - Network attacker can control page
- IE: displays mixed-content dialog to user
 - Flash files over HTTP loaded with no warning (!)
 - Note: Flash can script the embedding page
- Firefox: red slash over lock icon (no dialog)
 - Flash files over HTTP do not trigger the slash
- Safari: does not detect mixed content

Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting

FRAMES AND FRAME BUSTING

Frames

- Embed HTML documents in other documents

```
<iframe name="myframe"  
  src="http://www.google.com/">  
  This text is ignored by most browsers.  
</iframe>
```



Frames and frame busting

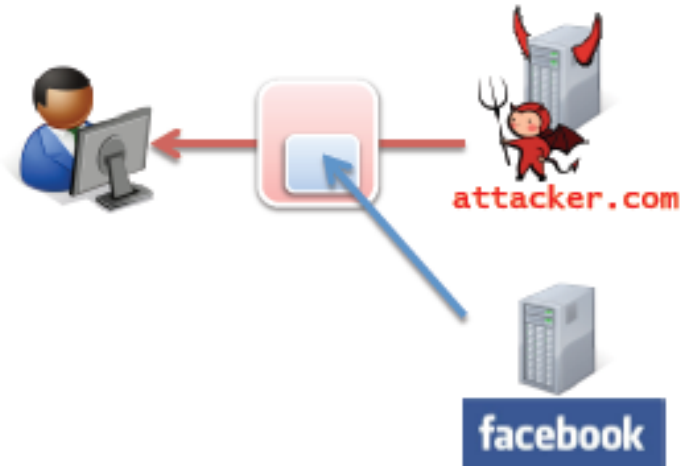
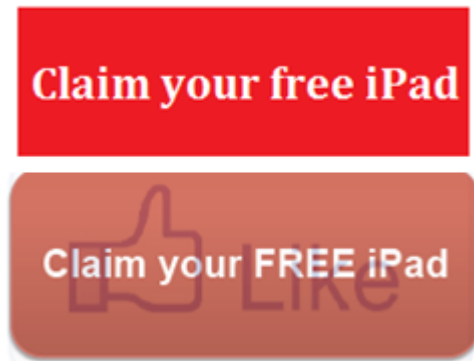
Clickjacking

How many of you recognize this?



Likejacking

- The user can be tricked into clicking  button, on an attacker's website
- User visits attacker.com
- Like button hidden behind another button



Clickjacking

- In a clickjacking attack
 - a user is lured into clicking a button
 - The button is not associated with the page displayed by the browser
- Attacker compromises context integrity of another app's UI components
 - Visual Integrity
 - Temporal Integrity

Types of Integrity

Visual Integrity

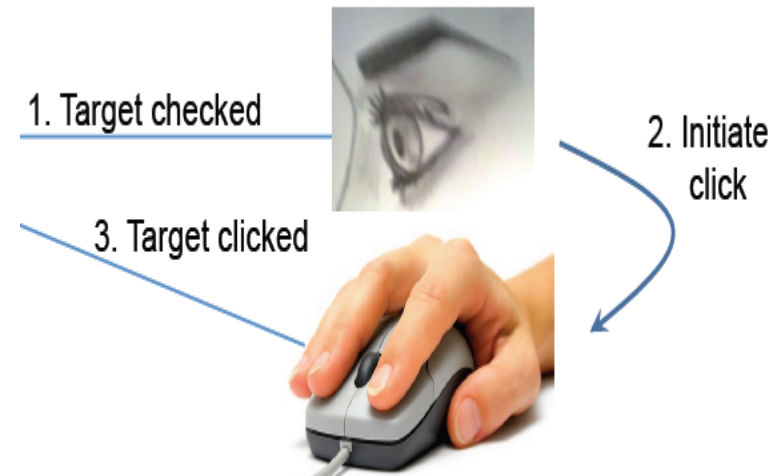
- What the user sees, is actually what is present
- No transparent, overlaid objects

- Eg  should be visible

should be visible

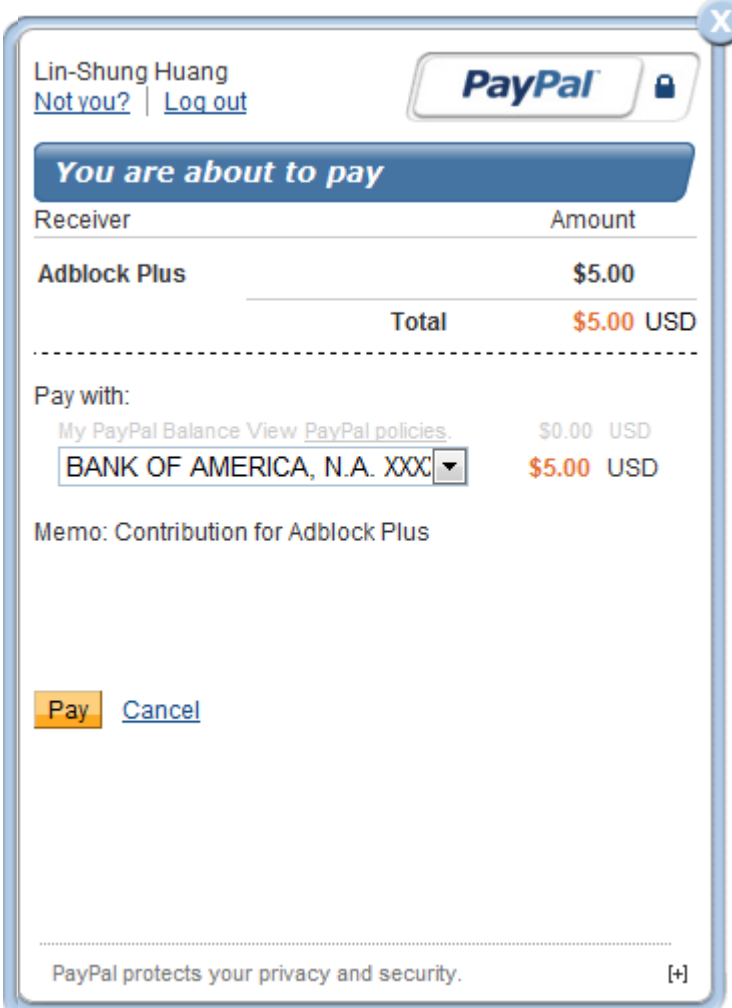
Temporal Integrity

- State of the UI between *time of user checking* and *the time of initiating the click*, remains the same



Compromising Visual Integrity

- Hide the target
- Partial Overlays



Lin-Shung Huang
[Not you?](#) | [Log out](#)

PayPal

You are about to pay

Receiver	Amount
Adblock Plus	\$5.00
Total	\$5.00 USD

Pay with:

[My PayPal Balance](#) [View PayPal policies](#) \$0.00 USD

\$5.00 USD

Memo: Contribution for Adblock Plus

[Cancel](#)

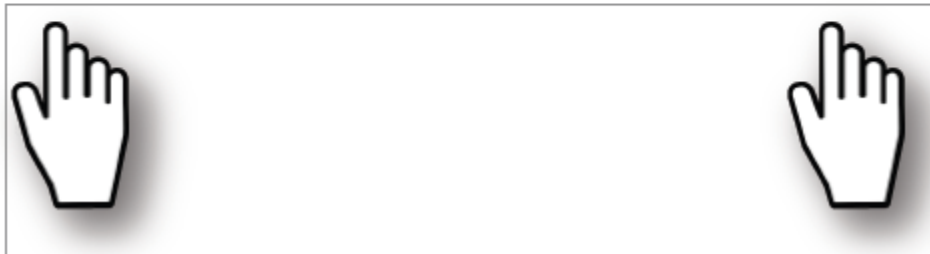
PayPal protects your privacy and security. [\[+\]](#)

Compromising Visual Integrity

- Multiple cursor feedback known as *cursorjacking*



Real Cursor



Fake Cursor



Compromising Temporal Integrity

- Bait and switch: As mouse comes near “Claim you..” button, Like moves to take it’s location before the user realizes it



Existing Defences

- User confirmation
 - Degrades user experience
- UI randomization
 - Unreliable & not user-friendly. (Multi-click attacks)
- Framebusting (X-Frame-Options)
 - Incompatible with embedding 3rd-party widgets
- Opaque overlay policy
 - Breaks legitimate sites
- Visibility detection on click
 - Allow clicks only on elements that are visible

Existing Defences

- User confirmation
 - Degrades user experience
- UI randomization
 - Unreliable & not user-friendly. (Multi-click attacks)
- Framebusting (X-Frame-Options)
 - Incompatible with embedding 3rd-party widgets
- Opaque overlay policy
 - Breaks legitimate sites
- Visibility detection on click
 - Allow clicks only on elements that are visible

ClickJacking Example

```
<html>
  <head>
    <title>Clickjacking Times</title>
  </head>

  <body>
    <h1>Clickjacking Example</h1>
    <div style="z-index:2; position:absolute; top:0; left:0; width: 100%;
height: 100%">
      <iframe src="http://www.facebook.com/home.php?" id="frame1"
style="opacity:0; filter:alpha(opacity=0);" width="100%" height="100%" />
      </iframe>
    </div>
    <div align="right" style="position:absolute; top:0; left:0; z-index:1;
width: 100%; height:100%; background-color: white; text-align:left;">
      <p><input type="submit" value="Press Here" /><br />Press this button
for an iPhone</p>
    </div>
  </body>
</html>
```

Frame Busting: A defense

- Goal: **prevent** web page from loading in a frame
 - example: opening login page in a frame
 - example: clickjacking with transparent iframe
- Frame busting:

```
if (top !== self)
    top.location = self.location
```



- More:

<https://seclab.stanford.edu/websec/framebusting/framebust.pdf>

Frame Busting Code

```
<style> body { display:none;} </style>
<script>
  if (self == top) {
    document.getElementsByTagName("body")[0].style.display =
      'block';
  }
  else {
    top.location = self.location;
  }
</script>
```

More : <https://seclab.stanford.edu/websec/framebusting/>

HTTP Headers

- X-Frame-Options HTTP response header
 - DENY: This page cannot be framed
 - SAMEORIGIN: Only pages from the same origin may frame this page
 - ALLOW-FROM <URI> : Only allow this specific URI to frame this page
- https://www.owasp.org/index.php/Clickjacking_Defense_Cheat_Sheet

Outline

- Web security model
- How browser works
 - Rendering content
 - Isolation
 - Communication
 - Navigation
 - Security User Interface
 - Frames and frame busting